6 SQL Concepts Every Data Engineer Should Know

A practical study guide based on the Medium article summary, with SQL examples and interview-oriented notes.

Source article: 6 SQL Concepts Every Data Engineer Should Know — Towards Data Engineering

Purpose: This document summarizes the six concepts and provides adapted examples; it is not a verbatim reproduction of the article.

At a glance

#	Concept	Primary use
1	Window functions	Analytics while retaining individual rows
2	Common Table Expressions (CTEs)	Break complex queries into readable stages
3	Stored procedures	Package and automate database operations
4	User-defined functions (UDFs)	Reuse business logic in SQL
5	System catalogs and metadata	Discover schemas and automate pipelines
6	Procedural logic and dynamic SQL	Conditional workflows and runtime-built queries

1. Window functions

Window functions calculate across a set of related rows without collapsing the result to one row per group. Unlike GROUP BY, they preserve row-level detail.

Example: show each employee's salary alongside the average salary in their department.

```
SELECT
employee_id,
employee_name,
department_id,
salary,
AVG(salary) OVER (
PARTITION BY department_id
) AS department_avg_salary
FROM employees;
```

Important functions

- **ROW_NUMBER()** — assigns a unique sequence number within a window.
- **RANK() / DENSE_RANK()** — ranks rows; they differ in how they handle gaps after ties.
- **SUM() OVER()** — supports running totals and cumulative aggregates.
- **LAG() / LEAD()** — accesses a previous or following row.
- **AVG() OVER()** — calculates a window average without removing detail rows.

Data engineering uses: deduplication, latest-record selection, top-N per group, running totals, and change detection.

2. Common Table Expressions (CTEs)

A CTE names a query result for use by a following statement. CTEs make complex transformations easier to read, reason about, and debug.

```
WITH employee_totals AS (
SELECT employee_id, SUM(amount) AS total_sales
FROM sales
GROUP BY employee_id
),
top_employees AS (
SELECT employee_id, total_sales
FROM employee_totals
WHERE total_sales > 100000
)
SELECT e.employee_name, t.total_sales
FROM top_employees t
JOIN employees e ON e.employee_id = t.employee_id;
```

Remember: a CTE is not necessarily a physically materialized temporary table. Whether it is inlined or materialized depends on the database engine and query.

3. Stored procedures and database automation

A stored procedure packages SQL statements into a reusable database program. Procedures can accept parameters and perform multiple operations, depending on the database.

Illustrative SQL Server example:

```
CREATE PROCEDURE LoadDailySales
@LoadDate DATE
AS
BEGIN
INSERT INTO fact_sales (sale_date, customer_id, amount)
SELECT sale_date, customer_id, amount
FROM staging_sales
WHERE sale_date = @LoadDate;
END;
```

Common uses:

- Loading staging data into fact tables.
- Running data-quality checks and writing execution logs.
- Packaging multi-step transformations.
- Supporting scheduled or application-triggered jobs.

Creating a procedure does not automatically execute it. A scheduler, application, or another database process must invoke it.

4. User-defined functions (UDFs)

UDFs encapsulate reusable logic that can be called from SQL expressions. Common categories include scalar functions, which return one value, and table-valued functions, which return a table-like result.

```
SELECT
order_id,
total_amount,
dbo.CalculateTax(total_amount, 8.25) AS tax_amount
FROM orders;
```

The example uses SQL Server-style function qualification. UDF syntax and capabilities vary by database. Centralizing logic can help consistency, but row-by-row execution may be costly in some engines; assess the query plan and platform behavior.

5. System catalogs and metadata

Metadata describes database objects such as schemas, tables, columns, data types, procedures, and functions. Inspecting metadata is a foundation for generic and metadata-driven pipelines.

```
SELECT
TABLE_SCHEMA,
TABLE_NAME,
COLUMN_NAME,
DATA_TYPE
```

```
FROM INFORMATION_SCHEMA.COLUMNS
ORDER BY
TABLE_SCHEMA,
TABLE_NAME,
ORDINAL_POSITION;
```

Practical applications:

- Discovering tables and columns automatically.
- Validating source and target schemas.
- Generating ingestion configurations.
- Detecting schema changes and building generic ETL/ELT jobs.

Metadata interfaces differ by engine. For example, SQL Server provides sys catalog views; other platforms expose their own information schemas and catalog tables.

6. Procedural logic and dynamic SQL

Procedural SQL introduces constructs such as IF/ELSE, loops, and variables. Dynamic SQL builds a query at runtime when table names, optional clauses, or other structural elements vary.

Illustrative SQL Server-style conditional load:

```
IF @LoadType = 'Full'
BEGIN
TRUNCATE TABLE fact_sales;

INSERT INTO fact_sales
SELECT *
FROM stg_sales;
END
ELSE
BEGIN
-- Apply incremental-load logic here
SELECT *
FROM stg_sales;
END;
```

Security rule: never concatenate untrusted values directly into SQL. Use parameterized execution (for example, sp_executesql in SQL Server) for values. Dynamic identifiers usually need validation or allowlisting.

Engineering considerations: define full versus incremental behavior carefully, handle late-arriving data, make retries safe, and consider transaction boundaries and audit logging.

Suggested learning order

Priority	Topic	Reason
1	CTEs	Build readable, staged transformations
2	Window functions	Solve analytical and row-level problems
3	Metadata and catalogs	Automate discovery and schema validation
4	UDFs	Reuse business rules where appropriate
5	Stored procedures	Package database operations
6	Procedural and dynamic SQL	Handle conditional and metadata-driven workflows

Data engineering interview extensions

For senior data engineering interviews, complement the six SQL concepts with the execution and reliability topics below.

- **Query performance:** execution plans, indexes, join strategies, partition pruning, and statistics.
- **Distributed SQL:** Spark SQL shuffles, partitioning, skew, and predicate pushdown.
- **Incremental processing:** watermarks, CDC, upserts, late-arriving records, and idempotency.
- **Data quality:** uniqueness, null checks, referential integrity, reconciliation, and auditability.
- **Schema evolution:** detecting and managing source schema changes safely.

Prerequisites checklist

- SELECT, WHERE, ORDER BY, GROUP BY, HAVING
- INNER / LEFT / RIGHT joins and join cardinality
- CASE expressions and NULL handling
- UNION versus UNION ALL
- Subqueries and aggregate functions

Source and scope

Based on the concepts and summary of the article “6 SQL Concepts Every Data Engineer Should Know”. Examples in this guide are adapted for learning; SQL syntax is database-specific and should be checked against the target engine.